Production Build Plan — Credit Risk Pipeline

Production Build Plan — Ambitious Path

A phase-by-phase roadmap to build a production-grade credit scoring pipeline that touches the tools recruiters look for: **Docker**, **Databricks**, **Snowflake**, **PySpark**, **Kubernetes**, **MLflow**, plus the supporting cast (FastAPI, Great Expectations, Prefect, GitHub Actions). Eight phases (A–H), each producing a portfolio-grade deliverable.

This is the longer sibling of `LEARNING_PLAN.md`. Where that plan optimised for time (4–5 weekends, minimal substitutes), this one optimises for **CV-truthful experience with the named tools**. ~12–16 weekends, deeper, more transferable.

Target audience: you, evenings and weekends. Local-only — zero cloud spend except for free trials of Snowflake and Databricks Community Edition.

Tech stack overlay — what tool, when, why

Every tool you'll touch, the phase it lands in, and the production concept it teaches.

Tool	Phase	Production concept it teaches	Local cost
uv + `pyproject.toml` + lockfile	A	Deterministic, reproducible builds	Free
pre-commit + ruff + black	A	Local quality gate; fast feedback loop	Free
GitHub Actions	A	Automated CI; no human is the merge gate	Free
pydantic-settings	A	12-factor config; same code, different envs	Free
Docker (single image)	B	Immutable infrastructure; "works on my machine" eliminated	Free
DuckDB	C	Warehouse semantics on a laptop; SQL portable	Free

Tool	Phase	Production concept it teaches	Local cost
		to Snowflake	
Great Expectations	C	Data contracts at the raw → staging boundary	Free
Snowflake (free trial)	C	Real warehouse; same SQL as DuckDB; time-travel	$400 trial credits / 30 days
MLflow (server)	D	Experiment tracking; model registry; lineage	Free
Docker Compose	D	Multi-service stacks; container networking	Free
PySpark	E	Distributed feature engineering; ML Pipelines	Free
SHAP	E + F	Reason codes for explainability	Free
FastAPI + Pydantic	F	Real-time inference services; OpenAPI contracts	Free
Uvicorn	F	ASGI server hosting	Free
minikube + kubectl	G	Local Kubernetes; Deployments, Services, HPA	Free
Helm (light touch)	G	Templated k8s manifests	Free
Prometheus + **Grafana**	H	Metrics scraping + dashboards	Free
Databricks (Community Edition)	H	Managed Spark + MLflow; enterprise UX	Free (limited)
Auto-generated model cards	H	SR 11-7-aligned governance documentation	Free

What you deliberately do NOT install: AWS / Azure / GCP CLIs, Terraform, OAuth providers, real Postgres, Nginx, Kafka, Airflow. Each is important in production and each is a rabbit hole that adds weeks without adding insight at this stage.

How the stack grows, phase by phase

Each box is a piece of running software on your laptop. The diagram shows what exists *at the end* of each phase — every later phase still has all the earlier boxes.

```
PHASE A — FOUNDATIONS  (DONE)

┌──────────────────────────────────────────────────────────────────┐
│  uv-managed venv  │  pyproject.toml + uv.lock  │  pre-commit hooks  │
│  GitHub Actions CI  │  pydantic-settings  │  Dockerfile (written)  │
└──────────────────────────────────────────────────────────────────┘

PHASE B — CONTAINERISE  (NEXT)

┌──────────────────────────────────────────────────────────────────┐
│  Phase A ↑                                                        │
│  + Docker image built ✓     + Docker volumes for data / models    │
│  + Pipeline runs in container ✓                                   │
└──────────────────────────────────────────────────────────────────┘

PHASE C — DATA LAYER

┌──────────────────────────────────────────────────────────────────┐
│  Phase B ↑                                                        │
│  + DuckDB warehouse (raw / staging / marts)                       │
│  + Great Expectations data contracts                              │
│  + Snowflake free trial (same SQL, same data)                     │
└──────────────────────────────────────────────────────────────────┘

PHASE D — EXPERIMENT TRACKING

┌──────────────────────────────────────────────────────────────────┐
│  Phase C ↑                                                        │
│  + MLflow server in Docker                                        │
│  + docker-compose orchestrates training + MLflow side-by-side     │
│  + Both models (scorecard + RF) registered with stages            │
└──────────────────────────────────────────────────────────────────┘

PHASE E — SPARK FEATURE ENGINEERING

┌──────────────────────────────────────────────────────────────────┐
│  Phase D ↑                                                        │
│  + PySpark feature pipeline                                       │
│  + Pre-split quantile leakage fixed via fitted transformer        │
│  + Feature pipeline serialised as MLflow artefact                 │
└──────────────────────────────────────────────────────────────────┘

PHASE F — REAL-TIME SERVING

┌──────────────────────────────────────────────────────────────────┐
│  Phase E ↑                                                        │
│  + FastAPI service with /v1/score, /healthz, /readyz, /metrics    │
│  + SHAP-based reason codes                                        │
│  + Serving Docker image (separate from training image)            │
└──────────────────────────────────────────────────────────────────┘

PHASE G — KUBERNETES

┌──────────────────────────────────────────────────────────────────┐
│  Phase F ↑                                                        │
│  + minikube cluster                                               │
│  + Deployment / Service / Ingress / ConfigMap / Secret / HPA      │
│  + Self-healing demonstrated (delete pod, watch new spawn)        │
└──────────────────────────────────────────────────────────────────┘
```

```
PHASE H — GOVERNANCE + DATABRICKS
┌──────────────────────────────────────────────────────────────────────┐
│  Phase G ↑                                                           │
│  + Auto-generated model cards                                        │
│  + Prometheus + Grafana dashboards (PSI, KS, latency)                │
│  + Same code lifted to Databricks Community Edition                  │
└──────────────────────────────────────────────────────────────────────┘
```

Phase A — Foundations

Goal: make the project reproducible, lint-checked, and CI-gated before anything else.

What you built

- `pyproject.toml` (uv-native dep contract, optional extras for later phases)
- `src/settings.py` (pydantic-settings — environment-driven config)
- `docker/training.Dockerfile` (multi-stage, written; not yet built)
- `.github/workflows/ci.yaml` (lint + test on every PR)
- `.pre-commit-config.yaml` (local hygiene hooks)
- `.env.example`, `.dockerignore`, extended `.gitignore`
- `tests/test_settings.py` (smoke tests)

Tools introduced: uv, pre-commit, GitHub Actions, pydantic-settings.

What you learn: deterministic builds, the difference between `requirements.txt` and a real lockfile, the three-module config split (`config.py` / `paths.py` / `settings.py`), why CI is a quality *gate* not a quality *report*.

Success criteria: `uv sync --group dev` works; `uv run pytest` passes; first push to GitHub triggers a green CI run.

CV line: *"Set up reproducible Python project with uv lockfile, pinned dependencies, automated CI/CD via GitHub Actions, and pre-commit quality gates."*

Time: done.

Phase B — Containerise the existing pipeline

Goal: the existing scorecard + RF training pipeline runs **inside a Docker container**, end to end, producing the

same outputs as bare metal.

What you'll build

- Build the `docker/training.Dockerfile` you already have written: `docker build -t credit-pipeline:dev -f docker/training.Dockerfile .`
- A `docker run` invocation that mounts `data/` and `models/` as volumes so the container can read your Excel and write back the trained model.
- A `Makefile` (or just documented commands in the README) so the build / run dance is one command, not three.
- An entry-point fix in `pipelines/run_pipeline.py` so it accepts a CLI arg (env-var or argparse) instead of a hardcoded path.

Tools introduced: Docker (image, container, volumes, layers, tags). This is the most foundational tool in the entire plan.

What you learn: what a container actually is (filesystem snapshot + isolated process — not a VM); why the Dockerfile order matters for layer caching; what a volume is and why you need volumes for any state that should survive a container restart; why training and serving images are usually separate; how the same image runs unchanged on a laptop, AWS ECS, GKE, anywhere.

Success criteria: stop your local Python interpreter. Run only this:

```
docker build -t credit-pipeline:dev -f docker/training.Dockerfile .
docker run --rm \
    -v $(pwd)/data:/app/data \
    -v $(pwd)/models:/app/models \
    -v $(pwd)/reports:/app/reports \
    credit-pipeline:dev -m pipelines.run_pipeline
```

You see the same scorecard + RF metrics print to your terminal that you'd see on bare metal. The trained scaler and any saved artefacts appear in `models/` on your host (because of the volume mount).

CV line: *"Containerised the credit scoring training pipeline with multi-stage Docker builds, separating builder and runtime layers for image size optimisation; demonstrated immutable infrastructure pattern."*

Time: one weekend. The first Dockerfile-against-real-code is always slower than you expect.

Phase C — Data layer (DuckDB + Great Expectations + Snowflake trial)

Goal: raw data lives in a SQL warehouse, not a single Excel file; every load is gated by a data contract; same SQL works against DuckDB locally and Snowflake in cloud.

What you'll build

- A `warehouse/credit.duckdb` file with three schemas: `raw`, `staging`, `marts`. Load the Excel into `raw.application` once.
- A Great Expectations suite at the `raw → staging` boundary: row count, primary-key uniqueness, target distribution, DRA range bounds, no-leakage check.
- An ingest module (`src/data/warehouse.py`) that reads from DuckDB, validates with GE, and writes to staging.
- A Snowflake free trial account + `account.toml` config. Deploy the same DDL there. Demonstrate one query running unchanged against both.

Tools introduced: DuckDB, Great Expectations, Snowflake (free trial).

What you learn: the lakehouse pattern (raw → staging → marts); time-travel as a reproducibility tool; why data contracts at the boundary catch upstream changes that would otherwise silently corrupt your model; SQL portability (and where it leaks — DDL is provider-specific, DML mostly isn't).

Success criteria: `python -m src.data.warehouse --ingest` loads, validates, and stages the data. A failing GE suite (e.g., raise the bad rate threshold past tolerance) blocks the load with a useful error. The same `SELECT` against `marts.applicant_features` returns identical results from DuckDB locally and from Snowflake.

Stretch: wire DuckDB *into the training container* via a volume mount, so Phase B's `docker run` reads from the warehouse instead of the Excel.

CV line: *"Designed a three-tier (raw / staging / marts) data warehouse pattern using DuckDB locally and validated against Snowflake for cloud parity; enforced data contracts via Great Expectations at ingestion boundary."*

Time: one to two weekends.

Phase D — Experiment tracking and model registry (MLflow)

Goal: every training run is logged; every model is versioned; nothing trained gets thrown away.

What you'll build

- `docker-compose.yml` with two services: `training` (your image from Phase B) and `mlflow` (the official MLflow image).
- An `src/training/train.py` that wraps the existing scorecard + RF logic in `with mlflow.start_run():` blocks, logging params, metrics, model, signature, input example.
- Both models registered: `credit_scorecard` and `credit_rf_challenger`. Stage the better one to `Production`.
- Auto-tagging: each run records git SHA, data snapshot, Python version, library versions.

Tools introduced: MLflow (tracking + registry), Docker Compose (multi-service stacks), Docker networks (training container talks to MLflow container).

What you learn: what an experiment is (a named collection of runs); what a *run* is (a single training execution + everything logged about it); how the registry separates *model versions* from *deployed stages*; why promotion to Production is an explicit action, not a side effect of training.

Success criteria: `docker compose up` starts MLflow. Running `docker compose run training` produces a run visible in the MLflow UI at `localhost:5000`. You can compare two runs side-by-side. You promote a model to Staging via the UI and the URI `models:/credit_scorecard/Staging` resolves.

CV line: *"Implemented experiment tracking and model lifecycle management with MLflow Model Registry; containerised the tracking server alongside training jobs via Docker Compose; promoted models through staged lifecycle (None → Staging → Production)."*

Time: one weekend.

Phase E — PySpark feature engineering

Goal: the same row-wise feature logic, ported to PySpark, with the pre-split quantile leakage fixed.

What you'll build

- A PySpark port of `src/features/features.py`. Every pandas operation gets a Spark equivalent (which is mostly mechanical at the row level).
- A custom `Estimator` / `Transformer` pair for the 70th-percentile quantile flag, fitted on training only,

joined into a `pyspark.ml.Pipeline`.
- The fitted Spark Pipeline saved as an MLflow artefact so it loads alongside the model at scoring time.
- The Spark image as an extras layer in your training Docker image: `uv sync --extra spark`.

Tools introduced: PySpark, Spark ML Pipelines, fitted feature transformers.

What you learn: the Spark execution model (lazy DataFrames, actions, the catalyst optimiser at a high level); why fitted transformers prevent leakage by design; how feature pipelines travel with the model; the difference between Spark on a single node (what you have) and Spark on a cluster (what Databricks gives you).

Success criteria: the Phase D training run produces the same metrics as before, but uses PySpark for feature engineering. The fitted feature pipeline is logged as `feature_pipeline/` under the MLflow run. Scoring a new record runs through the Pipeline (so train-time and inference-time transformations are bit-for-bit identical).

Stretch: run the Spark job inside its own container in `docker-compose.yml` (separate from the training service), demonstrating multi-container orchestration.

CV line: *"Implemented distributed feature engineering with PySpark including custom fitted transformers (Estimator/Transformer pairs) to prevent pre-split data leakage; serialised feature pipeline alongside model in MLflow registry."*

Time: one weekend.

Phase F — Real-time scoring service (FastAPI)

Goal: an HTTP service running in a Docker container returns a credit score and reason codes for any applicant payload, in under 200ms.

What you'll build

- `src/serving/app.py`: a FastAPI app exposing `POST /v1/score`, `GET /healthz`, `GET /readyz`, `GET /metrics`, `GET /model_info`.
- Pydantic request/response models derived from `src/config.py` so the API contract can never drift from the feature list.
- SHAP-based reason codes returned with every score.
- A separate `docker/serving.Dockerfile` (slim — only FastAPI + the model bundle, no Spark, no MLflow training deps).

- A `tests/test_scoring_api.py` contract test that hits the service and asserts response shape.

Tools introduced: FastAPI, Uvicorn, SHAP, OpenAPI auto-generated docs.

What you learn: what a real-time model service looks like; why the API contract is a versioned thing (the URL has `/v1/`); why model loading happens once at startup, not per-request; how FastAPI's dependency injection / Pydantic validation catches malformed payloads at the door.

This is your LEARNING_PLAN Phase 1 + Phase 2, collapsed. You're building the same scoring service the original plan envisioned, just on top of a more complete training/registry stack.

Success criteria: start the serving container. `curl -X POST localhost:8000/v1/score -d @sample_applicant.json` returns `{probability_of_default, score, reason_codes, model_version}`. Hit the Swagger UI at `localhost:8000/docs` and try it interactively.

CV line: *"Built a low-latency real-time scoring service with FastAPI, including SHAP-based explainability for adverse action notices; containerised separately from training for image-size optimisation; auto-generated OpenAPI documentation."*

Time: one to two weekends.

Phase G — Kubernetes deployment (minikube)

Goal: the FastAPI image from Phase F runs on a real Kubernetes cluster (locally), with self-healing, rolling updates, and horizontal autoscaling.

What you'll build

- A minikube cluster on your laptop (`minikube start --driver=docker`).
- `k8s/deployment.yaml` (3 replicas of the serving image, readiness probe on `/readyz`).
- `k8s/service.yaml` (ClusterIP exposing the deployment internally).
- `k8s/ingress.yaml` (HTTP entry point reachable from your laptop).
- `k8s/configmap.yaml` (MLflow URI, model name, log level).
- `k8s/secret.example.yaml` (template for credentials; real one stays out of git).
- `k8s/hpa.yaml` (autoscale 1–5 replicas based on CPU).
- A demo: load-test with `hey` and watch HPA scale up; `kubectl delete pod` and watch self-healing.

Tools introduced: Kubernetes (Deployment, Service, Ingress, ConfigMap, Secret, HPA), kubectl, minikube. Optionally Helm.

What you learn: the entire k8s vocabulary; why pods are ephemeral and stable addressing comes from Services; what "rolling update" actually means; how the HPA controller watches metrics and adjusts replica count; why this is the same shape of YAML you'd write for EKS / GKE / AKS.

This is your LEARNING_PLAN Phase 5. Same goal — deploy on a laptop k8s cluster — done with a cleaner stack underneath.

Success criteria: `kubectl get pods` shows 3 running. `curl <ingress>/v1/score` returns a score. `kubectl delete pod <one>` — the next `curl` still succeeds because another pod handles it; `kubectl get pods` shows the deleted pod replaced within seconds.

Honest warning: YAML is unforgiving. Indentation errors at 11pm are a rite of passage. Budget extra time for kubectl debugging.

CV line: *"Deployed real-time ML scoring service to Kubernetes (locally on minikube; manifests portable to EKS/GKE/AKS) including HorizontalPodAutoscaler, readiness probes, and rolling update strategy; demonstrated self-healing under simulated pod failure."*

Time: one to two weekends.

Phase H — Governance, dashboards, Databricks demo

Goal: the deliverable looks like a *product* a model risk reviewer would accept — model card, dashboards, the same training code running on Databricks.

What you'll build

- `src/governance/model_card.py` — auto-generates a per-version model card (markdown + PDF) from MLflow metadata, manual qualitative fields, and SHAP-based fairness analysis.
- Prometheus + Grafana in `docker-compose.yml`. Dashboards for:
    - Feature drift (PSI per feature, weekly)
    - Score drift (PSI on score distribution)
    - Discrimination (rolling Gini on scored-with-outcomes population)
    - Serving latency (p50, p95, p99)
    - Error rate (5xx ratio)
- A Databricks Community Edition notebook that runs the same Phase D training code unchanged. Proves the architecture is portable to a managed Spark + MLflow environment.

Tools introduced: Prometheus, Grafana, Databricks (Community Edition), model card generation.

What you learn: what governance artefacts a real bank or fintech model risk team expects; how monitoring goes from "logs in a file" to "queriable metrics with dashboards and thresholds"; how Databricks compares to a self-hosted MLflow + Spark setup (mostly: nicer UI, managed clusters, costs money).

Success criteria: open Grafana at `localhost:3000`, see five live dashboards. Generate a model card for the current Production model — it includes feature importance, calibration, segment-level performance. Open the Databricks Community Edition notebook, hit "Run All", see your model train end-to-end with MLflow runs visible in the Databricks UI.

CV line: *"Implemented SR 11-7-aligned governance including auto-generated model cards, drift monitoring (PSI / KS) on features and scores, and managed-runtime parity validation on Databricks Community Edition; built dashboards in Grafana sourcing Prometheus metrics."*

Time: one to two weekends.

Where you are now

```
A ▓▓▓
```

End of Phase F — repo layout. The training code and the serving code start to diverge in cadence. You can either keep them in one repo with subfolders (simpler for a portfolio), or split them into two repos (cleaner long-term, more work). For portfolio purposes, *one repo with two Dockerfiles* is the right answer.

End of Phase G — stop or push to cloud. Phase G ends with a fully working local stack. The natural extension is to deploy at least one slice into a real cloud (AWS App Runner is cheapest; Google Cloud Run is similarly cheap; both accept your exact Dockerfile). Optional, costs $5–20/month while you experiment.

End of Phase H — finish line or spin off? At the end of Phase H you have a complete production-grade pipeline. Option A: write it up as the centrepiece of your portfolio and start interviewing. Option B: peel off the FastAPI + Kubernetes piece into a separate `credit-scoring-service` repo so it stands alone as a deployable artefact (recruiters love single-purpose repos with clean READMEs).

Definition of done

You have built the full ambitious stack when:

1. `docker run credit-pipeline:dev` trains the model end-to-end with no Python installed on the host.
2. The pipeline reads from DuckDB locally; the same SQL queries Snowflake successfully.
3. Every training run is in MLflow; the registry has a Production-stage model.
4. Feature engineering runs on PySpark; the fitted feature pipeline is part of the model artefact.
5. A FastAPI container returns a credit score with reason codes from `localhost:8000/v1/score`.
6. The same image is deployed on minikube; you've demonstrated self-healing.
7. The same training code runs unchanged on Databricks Community Edition.
8. Every Production model has an auto-generated model card.
9. Grafana dashboards show feature PSI, score PSI, latency, and error rate.

When all nine are true, you can truthfully list **Docker, DuckDB / Snowflake, MLflow, PySpark, FastAPI, Kubernetes, Databricks** on your CV and back each one up with a concrete piece of code in this repo.

Rough total time

Phase	Time	Cumulative
A — Foundations	done ✓	0

Phase	Time	Cumulative
B — Containerise	1 weekend	1 weekend
C — Data layer	1–2 weekends	2–3 weekends
D — MLflow	1 weekend	3–4 weekends
E — PySpark	1 weekend	4–5 weekends
F — FastAPI	1–2 weekends	5–7 weekends
G — Kubernetes	1–2 weekends	6–9 weekends
H — Governance + Databricks	1–2 weekends	7–11 weekends

Realistic budget: **8–12 focused weekends** if you hit no major snags. Double it if you're also working a full week — production tooling breaks for reasons unrelated to ML, and debugging YAML at 11pm is a rite of passage.

If 12 weekends sounds long: it is. The trade-off is that at the end you have *concrete experience* with every named tool, not just a passing reference. That's the difference between getting screened past a recruiter filter and getting screened out.

Next action

Phase B — install Docker Desktop, build your first project image, run the existing training pipeline inside the container.

Step-by-step instructions for that are in chat. Once `docker run hello-world` works on your machine and Phase A verification is green, we start Phase B.